	**Carátula para entrega de prácticas**

Laboratorio de computación salas A y B

Profesor:	René Adrián Dávila Pérez	
Asignatura:	Programación Orientada a Objetos	
Grupo:	7	
No de Práctica(s):	10	
Integrante(s):	120002850	323064039
	323071378	323214636
	426059576	426058744
Semestre:	2027-1	
Fecha de entrega:	11-Septiembre-2026	
Obervaciones:		

CALIFICACIÓN: _____

Índice

1. Introducción **3**
1.1. Planteamiento del problema 3
1.2. Motivación 3
1.3. Objetivos 3

2. Marco Teórico **3**
2.1. Jerarquía fundamental: Throwable, Error y Exception 4
2.2. Excepciones verificadas y no verificadas 4
2.3. Excepciones en interfaces gráficas de usuario (GUI) 5
2.4. Reglas de orden y especificidad de bloques catch 5

3. Desarrollo **6**
3.1. Investigación del Ejercicio 1A: Jerarquía Error en Java 6
3.1.1. Síntesis académica de la jerarquía Error 6
3.2. Investigación del Ejercicio 1B: Excepciones comunes en GUI en Java . 11
3.2.1. Síntesis académica de excepciones en interfaces gráficas 11
3.3. Arquitectura y clases de la solución (Ejercicio 2) 16
3.4. Algoritmos, dinámica de ejecución y decisiones de flujo 18

4. Resultados **20**
4.1. Caso 1: Representativo (Éxito en límite válido de arreglo) 21
4.2. Caso 2: Frontera (Índice negativo fuera de rango) 21

5. Conclusiones **22**

1. Introducción

1.1. Planteamiento del problema

En el desarrollo de software orientado a objetos sobre la plataforma Java, la ocurrencia de eventos anormales e imprevistos durante el tiempo de ejecución constituye una de las principales causas de degradación y terminación prematura de los sistemas computacionales. La arquitectura de Java clasifica dichas condiciones mediante una jerarquía formal en la que convergen fallos críticos e irrecuperables de la máquina virtual y excepciones derivadas de la lógica del programa, del procesamiento de datos en arreglos o de la interacción asíncrona con el usuario en interfaces gráficas. Sin embargo, surge el problema metodológico y técnico de estructurar los mecanismos de detección y recuperación de manera coherente, puesto que una especificación incorrecta o invertida en el orden de captura polimórfica de excepciones genera código inalcanzable y comportamientos ambiguos, mientras que la falta de previsión en componentes visuales compromete la estabilidad del hilo de despacho de eventos.

1.2. Motivación

La construcción de aplicaciones robustas, confiables y tolerantes a fallos requiere que el ingeniero de software comprenda con profundidad la semántica de la jerarquía de excepciones y aplique principios deterministas en el control de anomalías. En los entornos gráficos interactivos, la captura puntual y disciplinada de eventos erróneos previene el congelamiento de la interfaz de usuario y garantiza la integridad del estado del programa sin interrumpir la experiencia de interacción. Asimismo, el tratamiento riguroso de excepciones asociadas a estructuras fundamentales de almacenamiento, como los arreglos, mediante bloques jerárquicos ordenados de lo particular a lo general, asegura una gestión predecible de recursos y consolida las competencias prácticas necesarias para diseñar sistemas orientados a objetos con altos estándares de calidad.

1.3. Objetivos

El objetivo fundamental de esta práctica consiste en analizar, contrastar y documentar la arquitectura de manejo de excepciones en Java SE, caracterizando formalmente las clases constitutivas de la jerarquía `Error` y las excepciones más comunes en el desarrollo de interfaces gráficas mediante consultas estandarizadas a herramientas de inteligencia artificial generativa, así como validar experimentalmente el principio de especificidad en el ordenamiento de cláusulas `catch` sobre manipulaciones de arreglos, distinguiendo con precisión los límites entre fallos irrecuperables del sistema y condiciones excepcionales gestionables.

2. Marco Teórico

El tratamiento de excepciones en Java se fundamenta en un modelo de propagación tipada y control de flujo alternativo gobernado por clases que heredan de la raíz univer-

sal `java.lang.Throwable` [1]. La especificación del lenguaje y de la máquina virtual organiza este árbol en dos ramas disjuntas principales: la clase `java.lang.Error` y la clase `java.lang.Exception`.

2.1. Jerarquía fundamental: Throwable, Error y Exception

La clase `Throwable` concentra la funcionalidad inherente a la captura del estado de los registros y de la pila de llamadas (*stack trace*) en el instante preciso de la instanciación de la anomalía, permitiendo diagnosticar la secuencia de invocaciones activas. A partir de ella se derivan dos ramificaciones conceptualmente opuestas:

- **La jerarquía `Error`:** Conforme a la definición de Lindholm et al. [2], comprende anomalías graves originadas usualmente en el entorno del sistema operativo o en el funcionamiento interno de la Máquina Virtual de Java (JVM). Se trata de circunstancias catastróficas tales como el agotamiento de memoria física, la saturación del espacio de pila o la inconsistencia binaria de clases cargadas dinámicamente. Bloch [3] señala taxativamente que las aplicaciones convencionales no deben intentar capturar ni recuperarse de instancias de `Error`, ya que no es posible garantizar la consistencia de los datos en memoria tras su detonación.

- **La jerarquía `Exception`:** Abarca aquellas condiciones anómalas que un programa bien concebido puede razonablemente anticipar y manejar para restituir el flujo ordinario o culminar tareas de manera ordenada.

2.2. Excepciones verificadas y no verificadas

Dentro de `Exception`, la plataforma establece una distinción determinante en tiempo de compilación entre excepciones verificadas (*checked*) y no verificadas (*unchecked*) [4]:

- **Excepciones verificadas:** Corresponden a todas las subclases de `Exception` salvo aquellas que descienden de `RuntimeException`. El compilador de Java impone de forma estricta la regla de declarar o capturar (*catch-or-declare*), exigiendo que el método que pueda originarlas declare una cláusula `throws` o las procese explícitamente mediante un bloque `try-catch`.

- **Excepciones no verificadas:** Engloban a `java.lang.RuntimeException` y sus descendientes, así como a todas las instancias de `Error`. Estas excepciones surgen primordialmente a raíz de defectos de programación, violaciones de contratos de algoritmos o manipulación impropia de tipos y referencias, por lo que el compilador no demanda su declaración obligatoria, delegando en el diseñador del software la responsabilidad de prevenirlas mediante comprobaciones previas.

2.3. Excepciones en interfaces gráficas de usuario (GUI)

El desarrollo de interfaces gráficas en Java con las bibliotecas estándar Swing y AWT opera bajo un paradigma reactivo orientado a eventos, coordinado de manera asíncrona por un hilo especializado denominado *Event Dispatch Thread* (EDT) [4]. Una anomalía no controlada que ocurre en el contexto del despachador de eventos compromete la respuesta visual del componente afectado. Entre las excepciones más recurrentes en este ámbito destacan:

1. `NullPointerException`: Invocación de operaciones sobre referencias nulas en componentes, oyentes de eventos o modelos de datos como `DefaultListModel` o `TableModel` que no han sido inicializados.

2. `IllegalArgumentException`: Provisión de argumentos con valores fuera de rango o incompatibles a administradores de esquemas de disposición (*LayoutManagers*) o métodos de dimensionamiento.

3. `IllegalStateException`: Modificación de componentes visuales en un momento incompatible con su ciclo de vida o alteración del estado de la interfaz fuera del hilo EDT, vulnerando la restricción de confinamiento de hilo (*thread confinement*).

4. `HeadlessException`: Intento de instanciar componentes visuales dependientes de teclado, ratón o monitor en plataformas de ejecución configuradas en modo *headless* (servidores sin pantalla o entornos de integración continua).

5. `IndexOutOfBoundsException`: Consulta o selección de posiciones inexistentes en elementos estructurados con índices, tales como pestañas en `JTabbedPane`, filas en `JTable` o elementos en `JList`.

6. `NumberFormatException`: Subclase de `IllegalArgumentException` provocada al intentar convertir el texto capturado desde un control `JTextField` hacia un tipo numérico cuando el usuario introduce caracteres no numéricos o cadenas vacías.

2.4. Reglas de orden y especificidad de bloques catch

La especificación oficial de Java [1] estipula que los bloques `catch` vinculados a una estructura `try` se evalúan secuencialmente de arriba hacia abajo. En virtud del principio de polimorfismo, una referencia de una superclase es capaz de enlazar cualquier objeto de sus subclases. Por consiguiente, si una cláusula `catch` que atrapa una superclase antecede a una cláusula que pretende capturar una subclase, esta última se vuelve inalcanzable. El compilador de Java detecta esta condición y rechaza el programa con un error de compilación por código inalcanzable (*unreachable catch block*). De este modo, la regla canónica exige ordenar los bloques de captura en sentido estrictamente decreciente de especificidad: primero las subclases particulares y al final las superclases generalizadas.

3. Desarrollo

3.1. Investigación del Ejercicio 1A: Jerarquía Error en Java

Para el Ejercicio 1A, la consulta común depositada en el repositorio bajo la ruta `Capturas/query_1A.txt` solicita una respuesta de 90 a 120 palabras, sin fórmulas de cortesía, explicando las clases principales de la jerarquía `Error` en Java (`VirtualMachineError`, `OutOfMemoryError`, `StackOverflowError`, `LinkageError`, `ExceptionInInitializerError`, `NoClassDefFoundError` y `AssertionError`), detallando su criticidad en la JVM y conservando ejemplos conceptuales breves.

Para el desarrollo experimental de esta sección, los seis integrantes del equipo ejecutaron consultas independientes recurriendo a dos de los principales ecosistemas de inteligencia artificial generativa: OpenAI ChatGPT (en modalidades de escritorio Web y aplicación móvil bajo iOS, implementadas por Alan, Zoé, Luis y Hanniel) y Google Gemini (en sus variantes Gemini 4 Argon y Gemini 3.6 Flash para Android, consultadas por Joseph y Alex). A través de la evidencia empírica documentada en las figuras 3.1 a 3.6, se constató cómo las directivas formuladas condicionaron la granularidad técnica de las respuestas. Mientras que las consultas estándar obtuvieron revisiones taxonómicas tradicionales fundamentadas en la raíz `java.lang.Throwable`, las instrucciones que incorporaron restricciones de síntesis visual o brevedad rigurosa detonaron esquematizaciones estructurales en árbol y puntualizaciones axiomáticas sobre la naturaleza irrecuperable de dichas anomalías.

3.1.1. Síntesis académica de la jerarquía Error

A partir del análisis de las respuestas empíricas y de las especificaciones canónicas de Java [1, 2], se caracterizan a continuación las principales clases pertenecientes a la rama `java.lang.Error`:

- **`VirtualMachineError`:** Clase abstracta base que agrupa los errores arrojados cuando la máquina virtual se encuentra en un estado degradado irrecuperable o ha agotado los recursos indispensables para continuar su ejecución.
- **`OutOfMemoryError`:** Se detona cuando el recolector de basura (*Garbage Collector*) no logra liberar memoria suficiente en el montículo (*heap*) para satisfacer una solicitud de reserva de un nuevo objeto.
- **`StackOverflowError`:** Emerge cuando una invocación recursiva sin condición de parada o una cadena desmedida de llamadas a métodos agota el espacio asignado a la pila de ejecución del hilo en curso.
- **`LinkageError`:** Clase base que señaliza que una clase presenta una dependencia binaria rota respecto a otra que sufrió modificaciones incompatibles con posterioridad al proceso de compilación original.
- **`ExceptionInInitializerError`:** Se produce de forma envolvente cuando ocurre una excepción no controlada dentro de un bloque de inicialización estático (`static {...}`) o durante la asignación inicial de un atributo estático.

- **`NoClassDefFoundError:`** Ocurre cuando la JVM o un cargador de clases (*ClassLoader*) procura cargar la definición de una clase que estaba presente en tiempo de compilación pero cuya definición en *bytecode* ya no se localiza en tiempo de ejecución.

- **`AssertionError:`** Se lanza cuando falla una comprobación lógica introducida mediante la palabra clave `assert`, indicando la transgresión de un invariante interno del programa.

El análisis comparativo entre ambos entornos de procesamiento de lenguaje natural revela una convergencia conceptual unánime con la especificación de la JVM [1, 2]. Ambos modelos catalogaron a la jerarquía `Error` como una rama no verificada (*unchecked*) orientada a fallas catastróficas del sistema operativo o del entorno de cómputo que escapan al control ordinario de la lógica de negocio, desaconsejando de manera unísona su intercepción mediante bloques `catch`. En las figuras 3.1 a 3.6 se presentan las evidencias directas de las consultas ejecutadas por Alan, Zoé, Luis, Joseph, Hanniel y Alex, validando empíricamente la solidez de las definiciones conceptuales obtenidas frente al estándar oficial de Java.

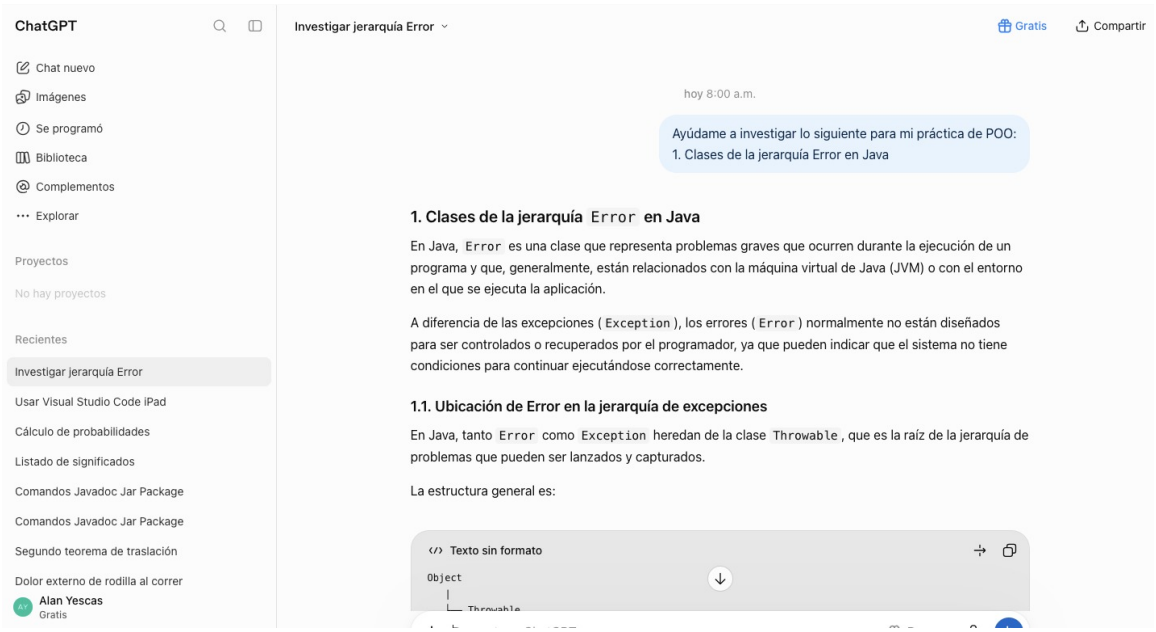


Figura 3.1: Consulta de Alan sobre la jerarquía Error en Java (Ejercicio 1A).

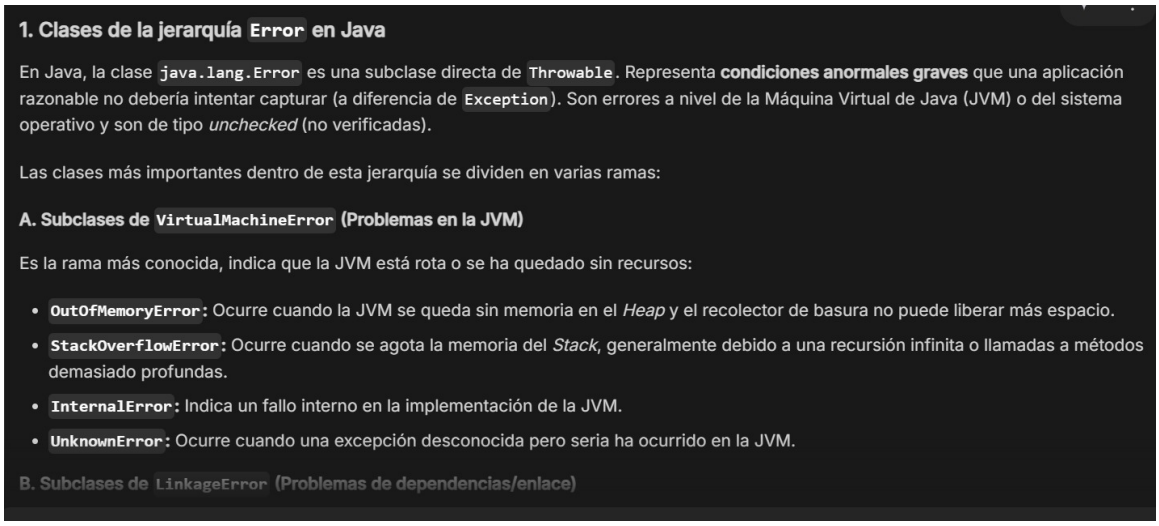


Figura 3.2: Consulta de Zoé sobre la jerarquía Error en Java (Ejercicio 1A).

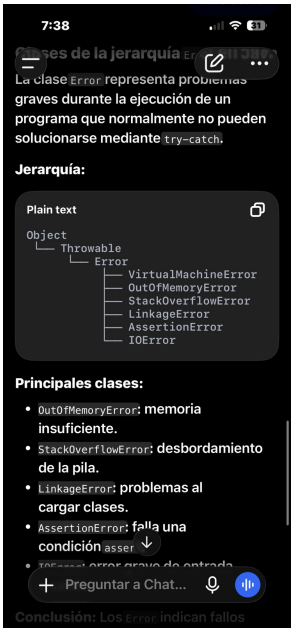


Figura 3.3: Consulta de Luis sobre la jerarquía Error en Java (Ejercicio 1A).

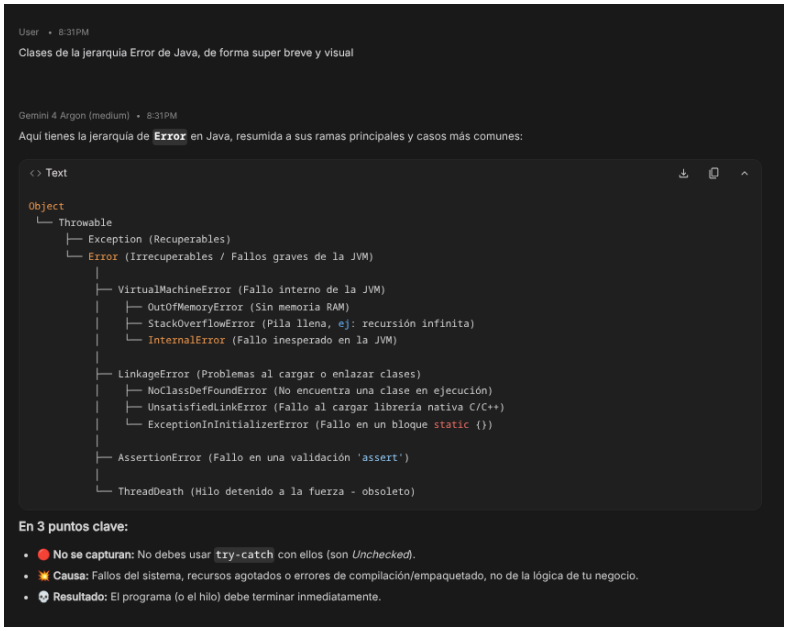


Figura 3.4: Consulta de Joseph sobre la jerarquía Error en Java (Ejercicio 1A).

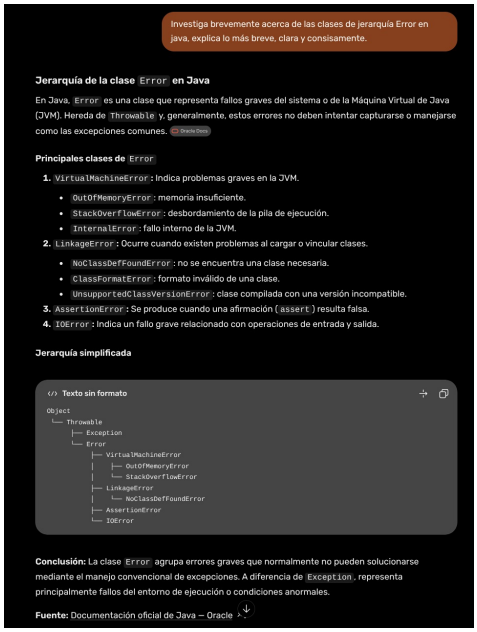


Figura 3.5: Consulta de Hanniel sobre la jerarquía Error en Java (Ejercicio 1A).

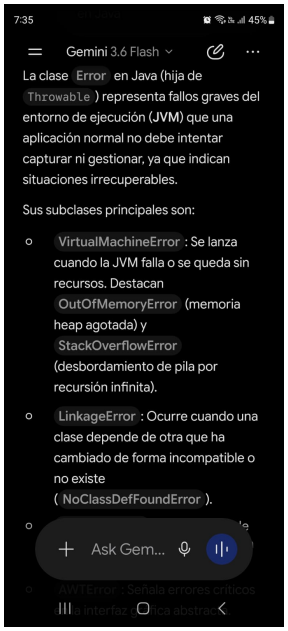


Figura 3.6: Consulta de Alex sobre la jerarquía Error en Java (Ejercicio 1A).

3.2. Investigación del Ejercicio 1B: Excepciones comunes en GUI en Java

Para el Ejercicio 1B, la consulta común depositada en el repositorio bajo la ruta `Capturas/query_1B.txt` solicita una respuesta de 90 a 120 palabras, sin fórmulas de cortesía, explicando las excepciones más recurrentes al programar interfaces gráficas de usuario (GUI) en Java con Swing y AWT: `NullPointerException`, `IllegalArgumentException`, `IllegalStateException`, `HeadlessException`, `IndexOutOfBoundsException` y `NumberFormatException`, indicando su contexto de interacción y conservando ejemplos breves de código obligatorios.

La verificación empírica del Ejercicio 1B se fundamentó en la contrastación de las consultas efectuadas por los seis integrantes del equipo utilizando diversas plataformas: OpenAI ChatGPT (empleado por Alan, Zoé y Luis en sus modalidades web y móvil), Google Gemini (utilizado por Joseph en su versión Gemini 4 Argon y por Alex en Gemini 3.6 Flash) y Anthropic Claude (empleado por Hanniel). A nivel metodológico, se observó una marcada diferenciación en las estrategias de inducción: mientras Alan, Zoé y Luis recurrieron a consultas directas del temario, Hanniel y Joseph adoptaron técnicas de encadenamiento conversacional solicitando estructuras sintéticas y visuales, y Alex impuso restricciones cuantitativas de longitud (*"en 100 palabras"*). A pesar de estas diferencias en la formulación de los *prompts*, las respuestas convergieron de forma consistente en la identificación de anomalías operativas críticas, como la ausencia de periféricos gráficos en entornos de servidor (`HeadlessException`) y la falta de inicialización previa de referencias en la jerarquía de contención (`NullPointerException`).

3.2.1. Síntesis académica de excepciones en interfaces gráficas

En los entornos gráficos de Swing y AWT, los flujos interactivos detonan situaciones anómalas que demandan un tratamiento preventivo y correctivo en la capa de presentación:

- **`NullPointerException`:** Ocurre al acceder a métodos de componentes visuales (como `setText` o `setVisible`) antes de completar su instanciación en el constructor o método de inicialización.
- **`IllegalArgumentException`:** Se manifiesta al asignar valores incongruentes a propiedades de presentación, por ejemplo dimensiones con coordenadas negativas o porcentajes fuera de los rangos válidos.
- **`IllegalStateException`:** Sucede cuando se altera la visibilidad o configuración de diálogos modales o ventanas en instantes prohibidos por el ciclo de renderizado, o al manipular componentes Swing fuera del hilo despachador EDT.
- **`HeadlessException`:** Ocurre al invocar diálogos interactivos mediante clases auxiliares como `JOptionPane` en entornos de servidor o microservicios donde no existe un servidor gráfico ni pantalla configurada.

- **`IndexOutOfBoundsException`:** Se produce al solicitar la remoción o lectura de una pestaña en `JTabbedPane` o una fila en `JTable` con un índice menor a cero o mayor o igual al total de elementos disponibles.
- **`NumberFormatException`:** Se genera de forma ubicua cuando el usuario introduce cadenas alfanuméricas o vacías en campos de entrada numérica (`JTextField`), y la aplicación ejecuta parseos directos mediante `Integer.parseInt` sin validación previa.

En el plano del rigor arquitectónico, los modelos profundizaron en la naturaleza concurrente de las interfaces de usuario en Java, advirtiendo explícitamente que la violación del hilo despachador de eventos (*Event Dispatch Thread* o EDT) detona `IllegalStateException` si se actualiza la interfaz gráfica fuera del método `SwingUtilities.invokeLater()`. Asimismo, se caracterizó el origen de `NumberFormatException` como una vulnerabilidad ubicua provocada por la falta de saneamiento de cadenas en campos de texto (`JTextField`), y se contextualizó `IllegalArgumentException` frente a especificaciones espaciales anómalas en los gestores de diseño. En las figuras 3.7 a 3.12 se presentan las evidencias documentales capturadas por cada estudiante, las cuales validan la transversalidad del manejo preventivo de excepciones en la capa de presentación.

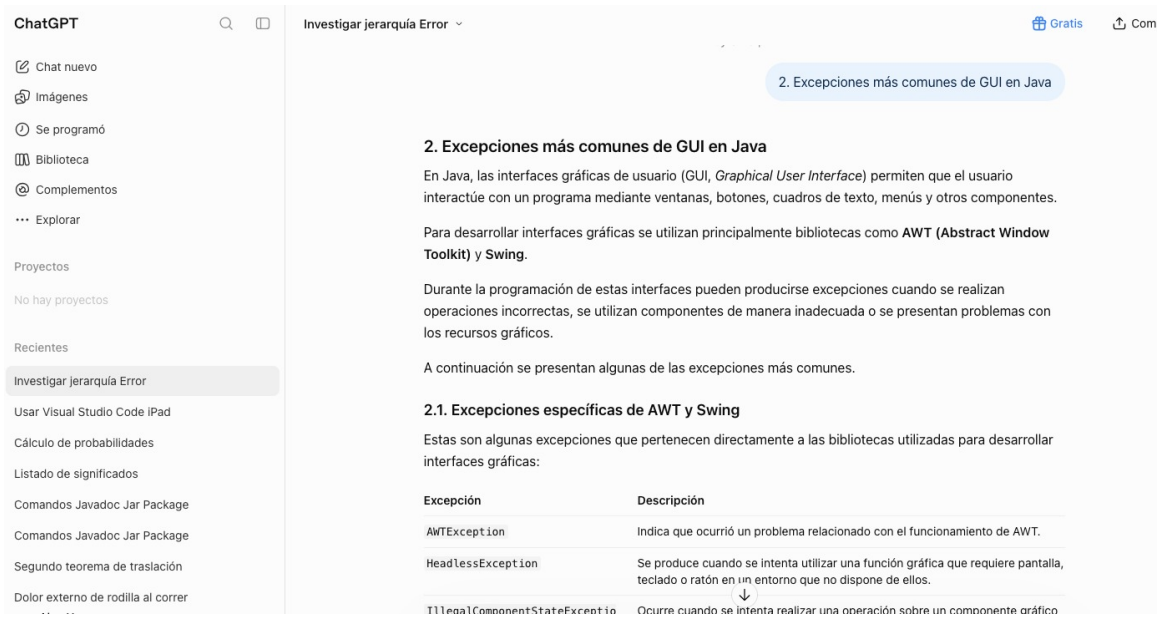


Figura 3.7: Consulta de Alan sobre excepciones comunes en GUI en Java (Ejercicio 1B).

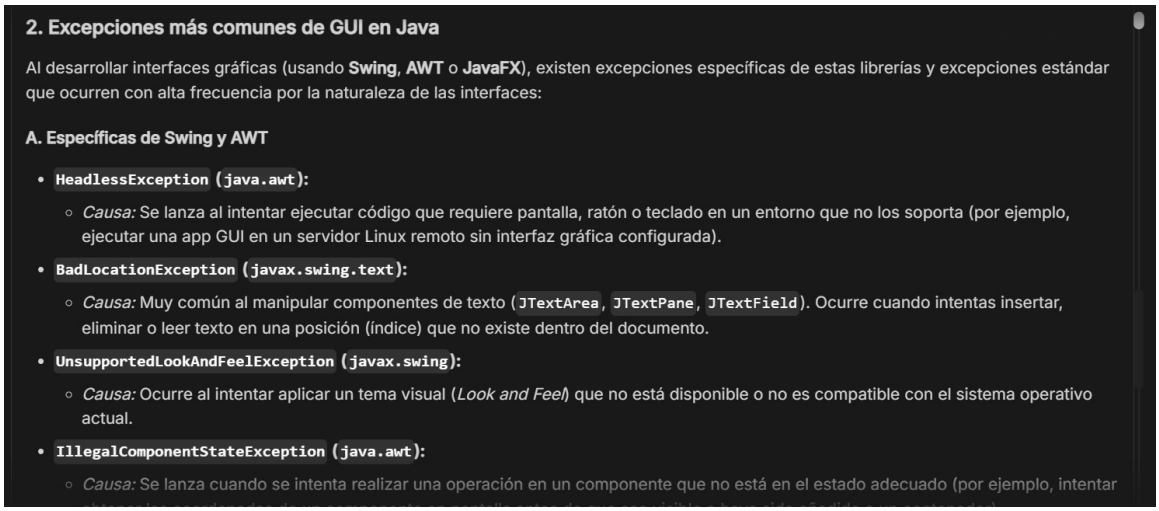


Figura 3.8: Consulta de Zoé sobre excepciones comunes en GUI en Java (Ejercicio 1B).

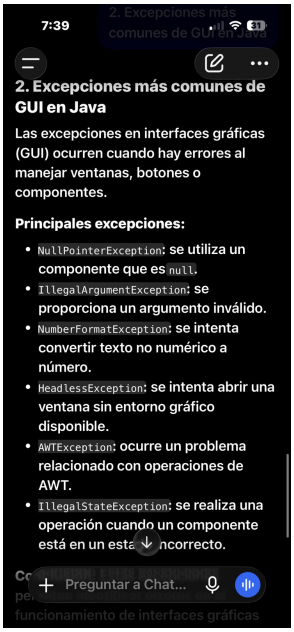


Figura 3.9: Consulta de Luis sobre excepciones comunes en GUI en Java (Ejercicio 1B).

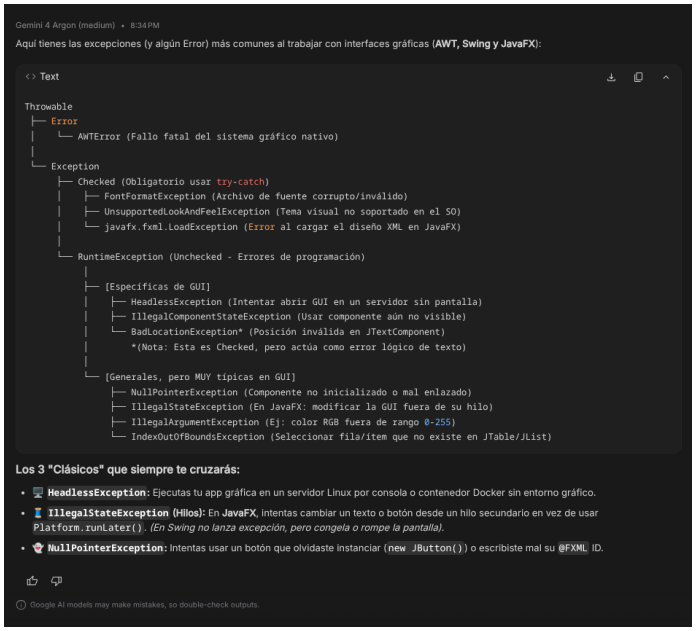


Figura 3.10: Consulta de Joseph sobre excepciones comunes en GUI en Java (Ejercicio 1B).

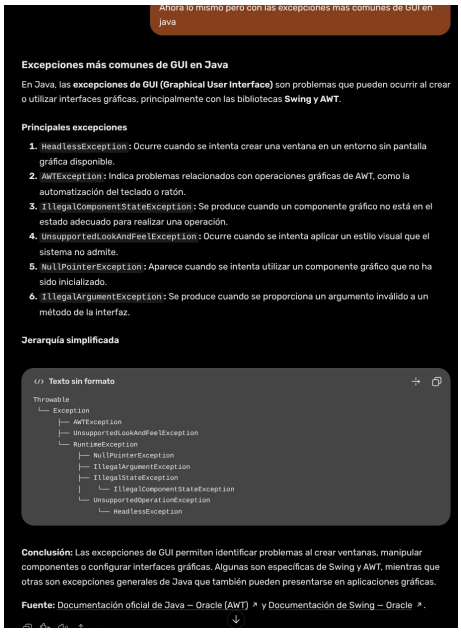


Figura 3.11: Consulta de Hanniel sobre excepciones comunes en GUI en Java (Ejercicio 1B).

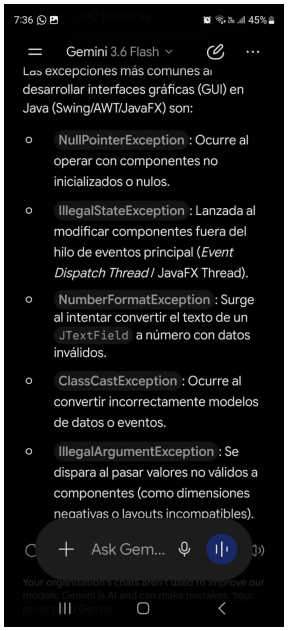


Figura 3.12: Consulta de Alex sobre excepciones comunes en GUI en Java (Ejercicio 1B).

3.3. Arquitectura y clases de la solución (Ejercicio 2)

Para evidenciar experimentalmente la importancia del orden de bloques `catch` tomando como modelo un arreglo y sus excepciones operativas más comunes, la arquitectura del sistema se organizó en dos clases modulares:

1. **Clase `ArregloSeguro`:** Encapsula un arreglo primitivo de enteros (`elementos: int[]`) y expone métodos de almacenamiento y verificación controlada. Protege el estado interno permitiendo parametrizar el dimensionamiento, la posición de escritura y la conversión desde cadenas de texto, registrando el comportamiento del evaluador jerárquico.

2. **Clase `DemostracionOrdenCatch`:** Clase controladora y punto de entrada que administra la interacción con el usuario a través de la consola estándar, orquestando tanto el caso ingresado como una batería integral de pruebas automatizadas para inducir sistemáticamente cada condición anómala.

La relación estructural entre ambas clases corresponde a una relación de composición fuerte de cardinalidad uno a uno (1 a 1), en la cual `DemostracionOrdenCatch` posee una instancia privada (`evaluador`) de tipo `ArregloSeguro`. La instancia del evaluador nace dentro del constructor del coordinador y su ciclo de vida se encuentra supeditado a él, asegurando coherencia referencial y encapsulamiento. En la figura 3.13 se detalla el diagrama de arquitectura global con la relación de composición ortogonal y sus multiplicidades explícitas. Asimismo, las especificaciones internas de cada entidad se presentan en las figuras 3.14 y 3.15.

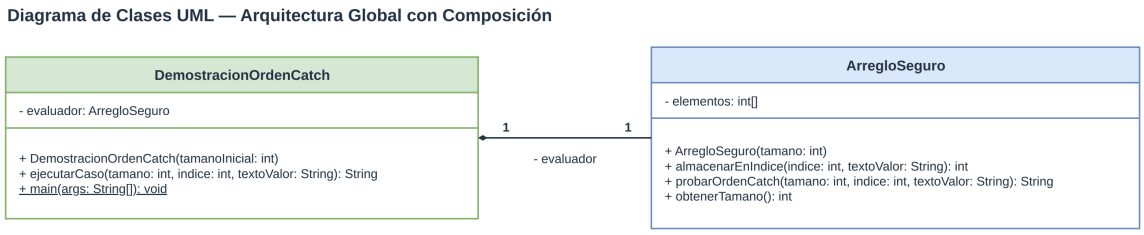


Figura 3.13: Diagrama de clases UML de arquitectura general mostrando la relación de composición entre DemostracionOrdenCatch y ArregloSeguro.

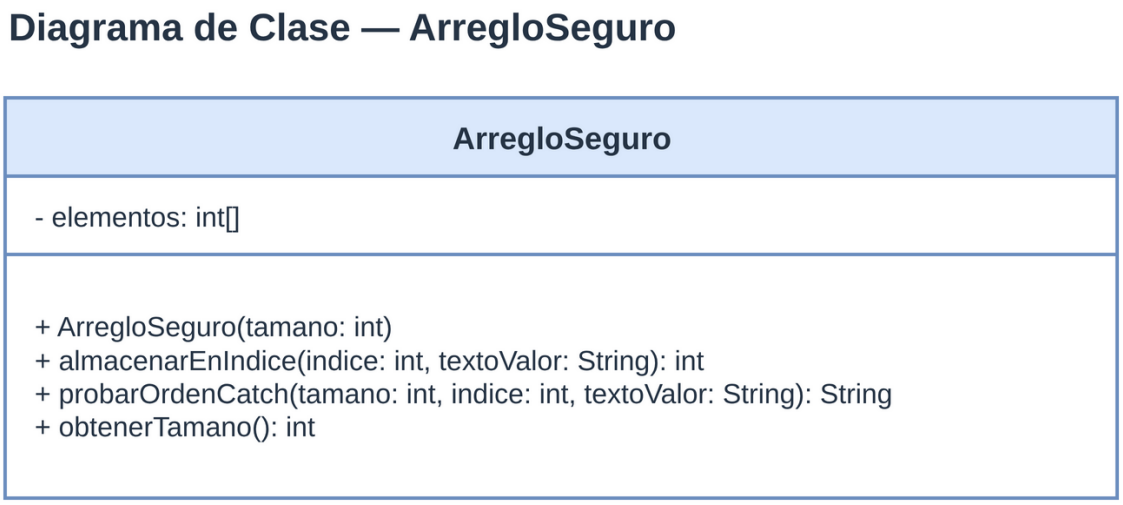


Figura 3.14: Diagrama de clase individual de ArregloSeguro detallando atributos privados y operaciones públicas.

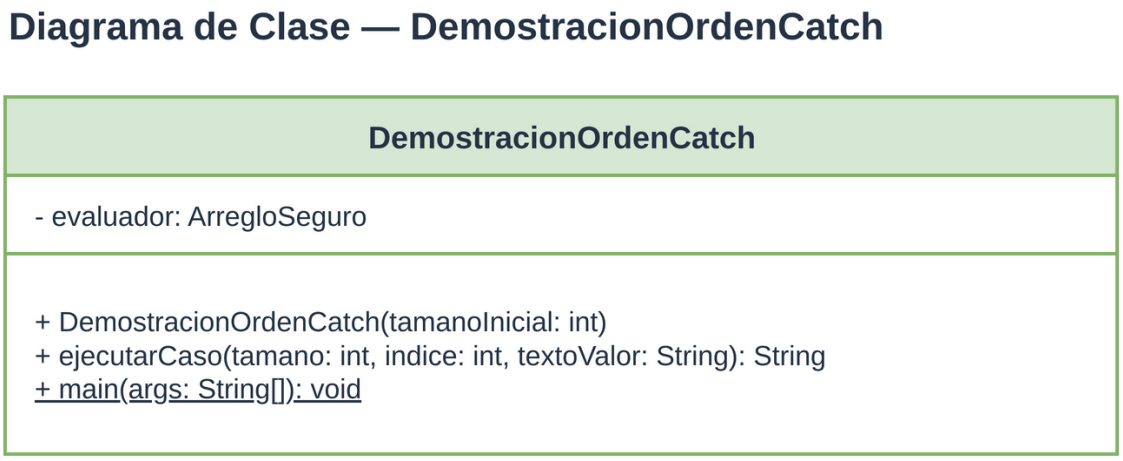


Figura 3.15: Diagrama de clase individual de DemostracionOrdenCatch con su atributo de composición y método principal.

3.4. Algoritmos, dinámica de ejecución y decisiones de flujo

El comportamiento dinámico se rige por la evaluación secuencial de tipos en tiempo de ejecución. En el método evaluador de `ArregloSeguro`, las operaciones de creación, conversión y acceso se encierran en una estructura de captura dispuesta de menor a mayor jerarquía:

1. **Capturas específicas prioritarias:** Se evalúan primero `ArrayIndexOutOfBoundsException` (para índices fuera de rango), `NegativeArraySizeException` (dimensiones negativas), `NumberFormatException` (entradas no convertibles) y `NullPointerException` (referencias nulas).

2. **Capturas intermedias y genéricas:** Posteriormente se disponen las clases intermedias `IndexOutOfBoundsException` y `RuntimeException`, reservando al final `Exception` y `Throwable` como salvaguardas de contención general.

Esta decisión de diseño asegura que cada eventualidad sea clasificada con su causa raíz más precisa. En la figura 3.16 se representa el diagrama de flujo correspondiente a la lógica de decisión y captura en `ArregloSeguro`. Por su parte, la figura 3.17 describe el flujo secuencial del método principal en `DemostracionOrdenCatch`, desde la lectura de parámetros hasta el reporte de resultados en consola. Finalmente, la figura 3.18 expone la colaboración temporal y el intercambio de mensajes entre el usuario, el despachador y el evaluador.

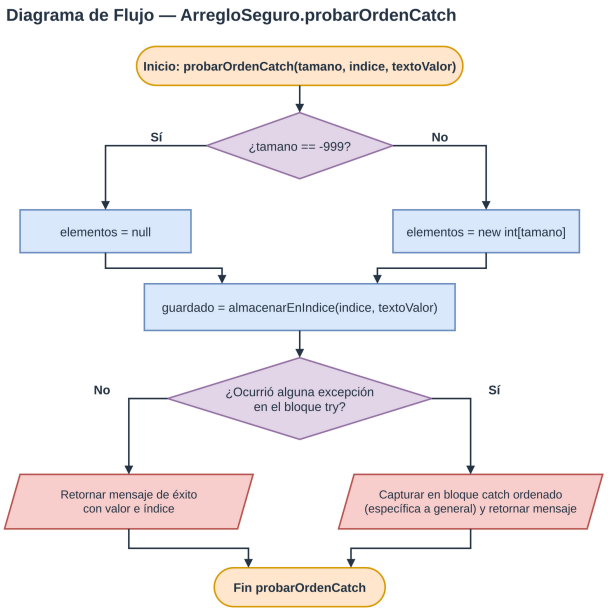


Figura 3.16: Diagrama de flujo del método probarOrdenCatch de la clase ArregloSeguro.

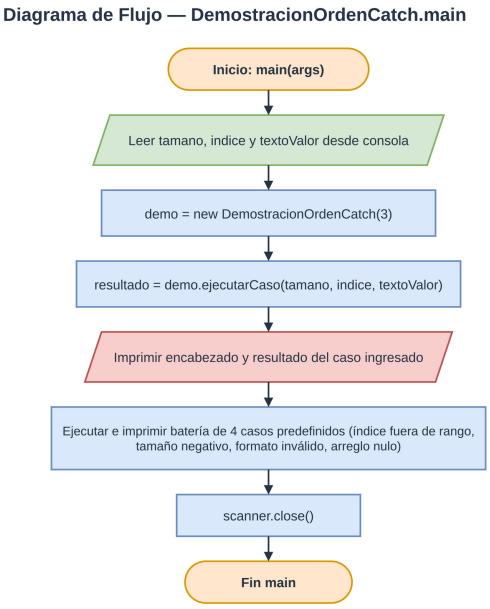


Figura 3.17: Diagrama de flujo del método principal de la clase DemostracionOrdenCatch.

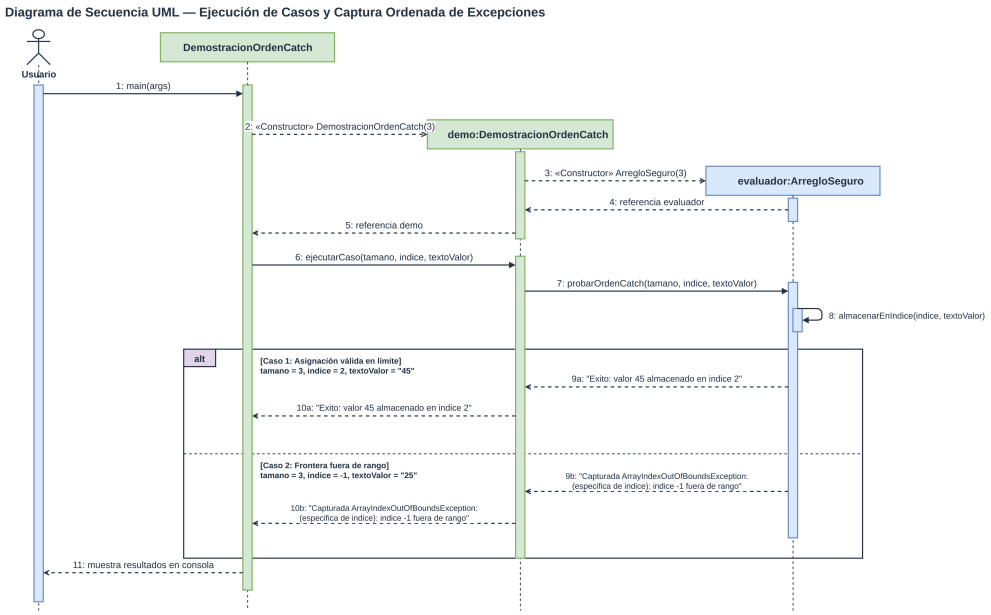


Figura 3.18: Diagrama de secuencia de la interacción temporal y captura de excepciones.

4. Resultados

La verificación empírica del sistema se efectuó mediante la ejecución de dos casos de prueba rigurosamente contrastados: un caso representativo en frontera válida y un caso frontera con violación de índice. En la tabla 1 se resume la matriz de condiciones analizadas, contrastando las entradas suministradas, el comportamiento esperado calculado y lo observado en consola.

Caso de prueba	**Entrada suministrada**	**Resultado observado / Excepción**	**Estado**
Caso 1 (Representativo en límite válido)	Tamaño: 3, Índice: 2, Valor: "45"	Almacenamiento exitoso en índice 2 y ejecución de batería de pruebas automáticas	Aprobado
Caso 2 (Frontera en índice negativo)	Tamaño: 3, Índice: -1, Valor: "25"	Captura específica de `ArrayIndexOutOfBounds-Exception` sin tocar bloques generales	Aprobado
Prueba automática 1	Tamaño: 3, Índice: 5	Captura determinista de `ArrayIndexOutOfBounds-Exception`	Aprobado
Prueba automática 2	Tamaño: -2	Captura puntual de `NegativeArraySize-Exception`	Aprobado
Prueba automática 3	Texto: .ªbc"	Intercepción de `NumberFormatException`	Aprobado
Prueba automática 4	Referencia nula	Detección de `NullPointerException`	Aprobado

Tabla 1: Matriz de casos de prueba y condiciones evaluadas en el manejo de excepciones.

Los resultados consolidados en la tabla 1 corroboran la correspondencia biunívoca entre las condiciones anómalas inducidas y las cláusulas de captura configuradas en el bloque de manejo de contingencias. La totalidad de las pruebas alcanzaron el estado de aprobación sin desbordar el ámbito de ejecución previsto ni generar fallas no controladas en el hilo principal. A continuación se desglosan en detalle los dos escenarios de ejecución interactiva capturados durante las pruebas del sistema.

4.1. Caso 1: Representativo (Éxito en límite válido de arreglo)

En el Caso 1 se evaluó la inserción en la posición extrema válida de un arreglo tridimensional ($N = 3$, índice 2). El comportamiento esperado calculado postulaba la correcta inserción del valor numérico 45 en la posición máxima admisible sin disparar excepciones, seguido de la activación secuencial de las cuatro pruebas automáticas complementarias. El comportamiento observado coincidió con exactitud total caracter por caracter, finalizando con código de salida 0. En la figura 4.1 se aprecia la captura de la consola ejecutada en el entorno de pruebas.

```
$ java -jar lib/practica10.jar
Tamano del arreglo: 3
Indice a escribir: 2
Valor entero en texto: 45
=== Demostracion del orden de bloques catch con un arreglo ===
Caso ingresado → Exito: valor 45 almacenado en indice 2
Caso 1 (Indice fuera de rango) → Capturada ArrayIndexOutOfBoundsException (especifica de indice): indice 5 fuera de rango
Caso 2 (Tamano negativo) → Capturada NegativeArraySizeException (especifica de tamano): tamano negativo -2
Caso 3 (Formato invalido) → Capturada NumberFormatException (especifica de formato): texto invalido "abc"
Caso 4 (Arreglo nulo) → Capturada NullPointerException (especifica de referencia): arreglo nulo
```

Figura 4.1: Captura de ejecución en consola del Caso 1 representativo demostrando inserción exitosa y capturas automáticas.

4.2. Caso 2: Frontera (Índice negativo fuera de rango)

En el Caso 2 se evaluó el comportamiento ante un índice frontera inválido consistente en un entero negativo (-1). El comportamiento esperado calculado exigía que la anomalía fuera interceptada de manera inmediata y unívoca por la cláusula específica `catch (ArrayIndexOutOfBoundsException e)`, impidiendo la propagación hacia `IndexOutOfBoundsException` o `Exception`. La salida observada en consola confirmó con exactitud dicha captura particular, conservando la estabilidad del proceso y ejecutando con normalidad los casos automáticos posteriores, tal como se documenta en la figura 4.2.

```
$ java -jar lib/practica10.jar
Tamano del arreglo: 3
Indice a escribir: -1
Valor entero en texto: 25
=== Demostracion del orden de bloques catch con un arreglo ===
Caso ingresado → Capturada ArrayIndexOutOfBoundsException (especifica de indice): indice -1 fuera de rango
Caso 1 (Indice fuera de rango) → Capturada ArrayIndexOutOfBoundsException (especifica de indice): indice 5 fuera de rango
Caso 2 (Tamano negativo) → Capturada NegativeArraySizeException (especifica de tamano): tamano negativo -2
Caso 3 (Formato invalido) → Capturada NumberFormatException (especifica de formato): texto invalido "abc"
Caso 4 (Arreglo nulo) → Capturada NullPointerException (especifica de referencia): arreglo nulo
```

Figura 4.2: Captura de ejecución en consola del Caso 2 frontera mostrando la intercepción por ArrayIndexOutOfBoundsException.

5. Conclusiones

El análisis de la arquitectura de excepciones en Java demuestra que el diseño orientado a objetos provee un mecanismo formal y tipado para separar la lógica de negocio habitual de las contingencias de tiempo de ejecución. La separación fundamental entre la jerarquía `Error` y la jerarquía `Exception` delimita con precisión la frontera entre las anomalías críticas del entorno de cómputo y la máquina virtual, cuya interrupción es ineludible, y aquellas circunstancias operativas recuperables que deben ser atendidas por el desarrollador para salvaguardar la disponibilidad del software.

Por otra parte, la comprobación experimental de la regla de especificidad en las estructuras `try-catch` ratifica que el principio de sustitución de Liskov y el polimorfismo exigen un ordenamiento estricto de menor a mayor generalidad en los manejadores de eventos anómalos. La comprensión de este ordenamiento, unida a la prevención de excepciones típicas en interfaces gráficas asíncronas sobre el hilo despachador de eventos, constituye una competencia fundamental para concebir programas estables, predecibles y tolerantes a fallos en el paradigma de programación orientada a objetos.

Finalmente, se constata que la robustez en la manipulación de estructuras de datos como los arreglos reside en combinar comprobaciones preventivas de invariantes con bloques de contingencia tipados. El uso prudente de excepciones no debe concebirse como sustituto del diseño por contrato ni del control algorítmico de precondiciones, sino como una barrera estructurada de contención que previene efectos secundarios indeseados y garantiza la resiliencia del sistema ante entradas no conformes.

Referencias

[1] Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A., Smith, D., & Bierman, G. (2023). *The Java Language Specification, Java SE 21 Edition*. Oracle America, Inc.

[2] Lindholm, T., Yellin, F., Bracha, G., Buckley, A., & Smith, D. (2023). *The Java Virtual Machine Specification, Java SE 21 Edition*. Oracle America, Inc.

[3] Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley Professional.

[4] Oracle Corporation. (2022). *Lesson: Exceptions. The Java Tutorials*. Oracle America, Inc.